

Computer Organization 2026 Programming Assessment III

Announce date: May 18, 2026

Exam date: May 28 & 29, 2026

0. Overview

This assessment aims to improve program execution performance through effective cache utilization. Modern computing systems leverage caches to reduce memory access latency by storing recently accessed or frequently used data closer to the processor. During the early stages of processor design, it is often the case that designers adjust cache organization and configurations according to application behavior. Later, when the processor is available, application code is typically optimized to match the hardware characteristics for improved performance. In this assessment, you will practice both of these concepts.

1. Application-aware cache design (Hardware perspective)

- Implement policies for managing limited cache resources
- Determine which data remains in cache and which gets evicted when cache is full to make room for new data

2. Cache-architecture aware memory access (Software perspective)

- Structure code to exploit cache behavior through spatial and temporal locality
- Organize memory access patterns to maximize cache hit rates

This programming assessment has a total score of **120 points**. You will be given public testcases to examine the correctness and performance of your implementations. Please note that there will be hidden testcases as well. Try to make your implementations handle the provided testcases properly. For details on how points are assigned, see ***section 5, Scoring Criteria***.

You will work on three problems.

- The first problem asks you to implement a cache replacement policy in the Spike simulator.
- The second problem asks you to optimize a matrix transpose program to reduce cache misses under a given cache configuration.
- The third problem asks you to optimize the matrix multiplication kernel used in a multilayer perceptron (MLP) inference program while preserving output correctness under a given cache configuration.

Suggested Workflow

1. Read this document carefully to understand the assessment requirements.
2. Complete Problem 1 by implementing the required cache replacement policy in the provided cache simulator files and then install them into your Spike.
3. Complete Problem 2 by optimizing the transpose implementation under the specified constraints.
4. Complete Problem 3 by optimizing the matrix multiplication kernel for the provided MLP inference program.
5. Test your code using the provided judge commands.
6. On the examination date in the computer classroom, you should follow the *Rules* and *Submission of Your Code* in *Section 7* to submit your work.

As our assessments are built upon open-source projects, unexpected compatibility issues may arise if you choose a different development environment. Our recommended development environment is provided as follows.

1. Environment Setup

Before you start, download the zip file for this assessment.

We recommend using the provided Docker image, which is the same environment used during the exam. It includes all required toolchains, `spike`, `pk`, and `valgrind` pre-installed.

If you prefer to build from scratch using your HW 0 toolchain, you must additionally install `valgrind` yourself, as **Problem 2 Matrix Transpose Optimization** depends on it.

Docker

A Docker environment will be provided during the examination in the computer classroom. Make sure you are familiar with basic Docker operations.

Create and start the container (before you run the command below, change your directory to where you extract the given zip file.)

```
$ docker run -it --name pa3 -v $(pwd)/C02026PA3:/workspace docker.io/asr1ab/comp-org:pa3
```

Check container status (In host system)

```
$ docker ps -a
```

Start container (In host system)

```
$ docker start pa3
```

Execute container (In host system)

```
$ docker exec -it pa3 bash
```

Exit container (In container)

```
$ exit
```

Stop container (In host system)

```
$ docker stop pa3
```

File Structure

The top-level directory of this assessment is organized as follows. Each sub-directory belongs to one problem in PA 3.

```
C02026PA3/  
├─ 1_cachesim/  
├─ 2_transpose/  
├─ 3_mlp/  
└─ Makefile
```

Makefile Commands

The top-level Makefile (shown in the File Structure) provides the following commands.

```
$ make judge-1 # judge 1_cachesim  
$ make judge-2 # judge 2_transpose  
$ make judge-3 # judge 3_mlp  
$ make judge-all # judge all problems  
  
$ make clean-1 # clean 1_cachesim  
$ make clean-2 # clean 2_transpose  
$ make clean-3 # clean 3_mlp  
$ make clean-all # clean all problems
```

Files That Students Can Modify

You are only allowed to modify the following four files:

- 1_cachesim/cachesim.h
- 1_cachesim/cachesim.cc
- 2_transpose/snippet.c
- 3_mlp/matmul_improved.c
- 3_mlp/dc_config.py

Any modifications to files other than the files listed above are not allowed and will be ignored.

2. Cache Optimization Fundamentals

Introduce Common Replacement Policies (Hardware Perspective)

As a cache replacement policy can greatly affect the cache hit/miss rate, it has been extensively studied. Consequently, many replacement policies have been proposed, each suitable for certain purposes. The cache hit/miss rate is an important metric for evaluating the efficiency of a cache design.

Three of the most commonly used cache replacement policies are listed below.

- **First In First Out (FIFO):** The cache behaves the same as a FIFO queue, where it evicts the cache blocks in the same order as they were added to the cache without considering any other factors.
- **Least Recently Used (LRU):** The least recently used cache blocks are discarded first.
- **Least-Frequently Used (LFU):** Cache blocks that are used least often are replaced first.

For more cache replacement policies (or the above three policies), please refer to the [Wikipedia](#) for more details.

Introduce Algorithmic Level Method to Reduce Cache Miss (Software Perspective)

The cache miss rate can be influenced by multiple factors. It is not only affected by the replacement policy but also by the way you implement the algorithm. Below, we list common strategies for reducing miss rates that you can apply to your algorithm. The major idea is to improve the spatial or temporal locality.

- **Loop Interchange:** Change the order you access data to improve the spatial locality on cache access.
- **Loop Fusion:** Combine the operation on same array at different loop to increase temporal locality.
- **Loop Tiling:** Break down a large loop into smaller blocks or tiles and organizing computations within each tile. Loop tiling enhances the reuse of data stored in the cache.
- **Packing:** Rearrange non-contiguous matrix data into a contiguous, cache-friendly format.

For more details, please refer to the links provided above.

3. Cache Simulation with Spike

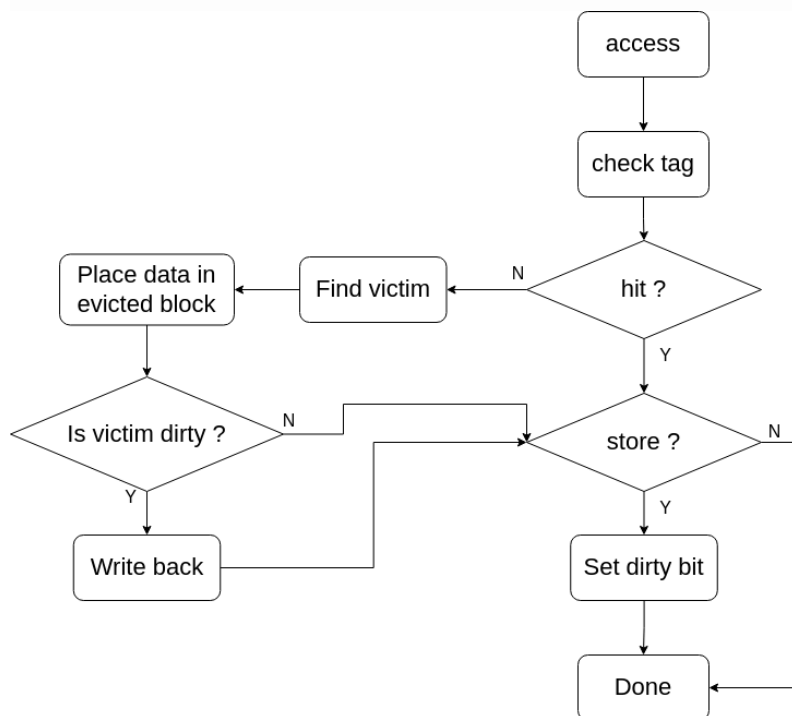
During the program emulation by Spike, the emulated instructions (the addresses of the instructions) are fed into the L1 *instruction cache simulator* and the data required by the emulated instructions (e.g., accessed via the memory instructions) are fed to the *L1 data cache simulator*. The cache simulator(s) are represented as the `cache_sim_t` class objects in Spike. The creation of the cache simulators is determined by command line parameters of Spike; for

example, the `dc` flag is used to create a L1 data cache simulator, and its parameters are used to specify the attributes of the created data cache, such as *set* and *ways*.

The `cache_sim_t` class defines the behaviors of the cache simulators created by Spike, where the prototypes are defined in `riscv/cachesim.h` and the implementation is in `riscv/cachesim.cc`. Both data and instruction caches inherit the interfaces defined in the `cache_sim_t` class. There are some important member functions related to manipulate the created cache(s), such as `access`, `check_tag`, and `victimize`. In the following paragraphs, we briefly introduce the high-level concepts of `access`, and `check_tag`.

Trace the source code to figure out more details of the member functions so as to understand the mechanism and workflow of the caches simulated by the `cache_sim_t` objects.

The workflow of the data cache simulation in Spike is highlighted as follows. The data cache simulation starts when a memory read/write is emulated by Spike, and the access function is invoked. The prototype of the access member function is `cache_sim_t::access(uint64_t addr, size_t bytes, bool store)`, where the parameter `addr` contains the address to the to-be-accessed data, `bytes` represent the size of the to-be-accessed data, and `store` indicates whether the access is a load or a store operation. The high level workflow is illustrated in the following image.



As for the `cache_sim_t::check_tag(uint64_t addr)` function, it is used to check the existence of a to-be-accessed data in the simulated data cache (i.e., check tag rectangle in the workflow image). The index to the set for the to-be-accessed data is calculated by the `addr` argument via the formula: $idx = (addr \gg idx_shift) \& (sets-1)$. If you have configured a four-set data cache, the `idx` value is the remainder of the dividing by 4 operation; that is, the value range of `idx` is from 0 to 3. Next, all of the tag in the corresponding ways are compared with the input

argument `addr` to find the existence of the data with the `addr` in the data cache. If the `addr` is found, it means a cache hit, otherwise it is a cache miss. When a cache miss occurs, the `victimize` function is invoked to find a victim data cache block to store the new input data. Currently, Spike implements only the random algorithm to randomly select the victim data block.

When cache miss occurs, the victim block should be selected to store the input data. In particular, Spike leverages the [Linear-Feedback Shift Register \(LFSR\)](#) to implement a pseudo-random cache replacement policy.

Note

Computer simulations do not always replicate real hardware specifications and behaviors. Instead, they focus on specific problems and use different workloads and simulation configurations to evaluate the trade-offs among different designs. For example, in this assessment, we focus on L1 data cache designs and aim to identify a cache design that results in the lowest data cache miss rate for the given workload. Furthermore, while cache miss penalties are considered in the evaluation of memory system overhead, the implementation overhead of different replacement policies (e.g., hardware area, power consumption, and computation latency) is not taken into account.

Performance Evaluation Methodology

The Spike simulator is used to assess the delivered performance of a given workload (application software) under a target RISC-V architecture design. In PA 3, the performance modeling is done by estimating the elapsed cycles on the CPU core (**InstCycle**) and the memory subsystem (**MemCycle**). Specifically, **InstCycle** is reported by the Spike simulator and is derived from the number of simulation instructions, whereas **MemCycle** is derived from the performance statistics reported by the cache simulation performed by Spike.

The **total exeuction cycle** of a given workload is estimated by the sum of **InstCycle** and **MemCycle**, where the **MemCycle** reports the data cache performance result only.

Improvement Ratio is used to quantify the performance improvement between different hardware or software configurations (e.g., code versions or cache configurations). The ratio can be calculated via the equation below.

$$\text{Improvement Ratio} = \frac{\text{InstCycle}_{\text{Original}} + \text{MemCycle}_{\text{Original}}}{\text{InstCycle}_{\text{Improved}} + \text{MemCycle}_{\text{Improved}}}$$

InstCycle_{Original} indicates the overall cycles spent on instruction execution in the original program, while **InstCycle_{Improved}** indicates the overall cycles spent on instruction execution in the improved version.

MemCycle_{Original} indicates the overall cycles spent on memory access in the original program, while **MemCycle_{Improved}** indicates the overall cycles spent on memory access in the improved

version.

To calculate the memory access overhead, you should use the formula below.

$$\text{MemCycle} = \text{cache}_{hits} \times \text{latency}_{hit} + \text{cache}_{misses} \times \text{penalty}_{miss}$$

The total cycles of memory access overhead are the summation of:

1. the number of cache hits: **cache_{hits}** multiplied by the latency for a cache hit access **latency_{hit}**, and
2. the number of cache misses: **cache_{misses}** multiplied by the cache miss penalty **penalty_{miss}**.

In this assessment, the data access latencies for a cache hit and a cache miss are configured as follows.

- Cache hit latency (**latency_{hit}**) = 1 cycle
- Cache miss penalty (**penalty_{miss}**) = 100 cycles

Example

The following table lists the performance data of an original version and the improved version.

	Original version	Improved version
InstCycle	214,406,569	250,725,004
MemCycle	3,103,225,866	110,471,856
D\$ Bytes Read	249965693	266092437
D\$ Bytes Written	2567727	18125391
D\$ Read Accesses	61817531	65755768
D\$ Write Accesses	373657	4216079
D\$ Read Misses	30644727	376695
D\$ Write Misses	72795	32396
D\$ Writebacks	81572	162280
D\$ Miss Rate	49.392%	0.585%

Based on the data and equations above, we can obtain the following data.

- **InstCycle_{Original} = 214,406,569**
- **MemCycle_{Original} = 3,103,225,866**
= ((61817531 – 30644727) + (373657 – 72795)) × 1 + (30644727 + 72795) × 100
- **InstCycle_{Improved} = 250,725,004**

- $\text{MemCycle}_{\text{Improved}} = 110,471,856$

$$= ((65755768 - 376695) + (4216079 - 32396)) \times 1 + (376695 + 32396) \times 100$$

$$\text{Improvement Ratio} = \frac{214406569 + 3103225866}{250725004 + 110471856} \approx 9.18$$

4. Problems

General Note

In this assessment, we assume the RISC-V ISA extensions are used, i.e., RV64GCV,

For all three problems, you are only required to modify the designated files listed in each problem statement.

Unless explicitly stated otherwise, you are **NOT** allowed to redesign the overall input/output flow of the provided framework. Testcase feeding, output generation, and judging are handled by the provided programs and scripts.

You should follow the provided framework and focus only on the required implementation/optimization tasks as specified below.

- **Problem 1 (Pseudo LRU algorithm, 10%):** Implement the pseudo LRU algorithm for cacahe replacement of the data cache simulator.
- **Problem 2 (Matrix Transpose Optimization, 40%):** Implement an optimized matrix transpose code to reduce cache misses under a fixed cache configuration.
- **Problem 3 (MultiLayer Perceptron; MLP, 70%):** Implementan optimized matrix multiplication code used in an MLP inference program.

I. Tree-based Pseudo LRU Cache Replacement Policy (weight: 10%)

In this problem, you will build a cache simulator by implementing the required replacement behavior in the provided source files. The simulator reads a sequence of memory accesses and produces hit, miss, and eviction results, along with summary statistics.

Background

Tree-based Pseudo LRU (Tree-PLRU) is a cache replacement policy that approximates true LRU with small hardware overhead. It represents each cache set using a binary tree of bits, where each internal node records which subtree is less recently used. By traversing these bits, the cache can efficiently choose a replacement candidate. After each access, the bits along the path are updated to point away from the accessed line. Although Tree-PLRU does not always match true LRU exactly, it is simple, efficient, and has been adopted in real processors such as the Intel 486 and some PowerPC CPUs. For more information, you may refer to [CS 3330 – Tree-PLRU](#) and [Wikipedia](#).

For the overall cache-simulation workflow and the role of functions such as `access` , `check_tag` , and `victimize` , please also refer to **Section 3, Cache Simulation with Spike**. Problem 1 focuses on implementing the replacement-policy part in the Spike simulator.

Files

The list of files involved with the first problem is shown below.

```
1_cachesim/
├── answer/      # reference answers
├── input/       # testcases
├── output/      # your output
├── cachesim.cc  # YOUR WORK HERE
├── cachesim.h   # YOUR WORK HERE
├── common.h
├── memtracer.h
├── csim.cc      # parse input sequence
├── judge.py
└── Makefile     # judge commands
```

You should **only** modify the following two files:

- `1_cachesim/cachesim.h`
- `1_cachesim/cachesim.cc`

Do **not** modify other source files, input files, judge scripts, or output formatting logic unrelated to the replacement policy.

These two files are inherited from `riscv-isa-sim/riscv/cachesim.h` and `riscv-isa-sim/riscv/cachesim.cc` , with some customizations for this assessment.

Implementation Requirements

To complete this problem, your simulator must:

1. maintain one independent Tree-PLRU state for each cache set,
 2. initialize every internal Tree-PLRU node to point to `LEFT` ,
 3. update the tree on every cache hit according to `UpdateTreeOnHit` ,
 4. select the target way on every miss according to `SelectVictimOnMiss` ,
 5. follow the **Tree-PLRU Rules** exactly as specified, even when invalid ways exist in the set.
- `LEFT` : if a node points to `LEFT` , it means the PLRU way is in the left subtree.
 - `UpdateTreeOnHit` : when a hit occurs, visit the tree from the hit leaf to the root, and update the direction of each visited parent node.
 - `SelectVictimOnMiss` : when a miss occurs, visit the tree from the root to a leaf according to the node directions, and update the direction of each visited internal node. The reached leaf is selected as the victim way.

For the exact behavior, please refer to ***Tree-PLRU Rules*** below.

Note

In this problem, all public and hidden testcases use set-associative cache configurations only.

You do not need to consider fully associative cache configurations when implementing your solution.

You only need to implement the replacement policy in the provided cache simulator framework. Input parsing, operation handling, and output formatting are already managed by the existing code.

Tree-PLRU Rules

Basic Setting

```
LEFT  = 0
RIGHT = 1
```

Initially, all non-leaf nodes point to LEFT.

Each cache set maintains its own independent Tree-PLRU state.

Leaves are mapped to cache ways from left to right.
For example, in a 4-way set:



Pseudo Code

```
PROCEDURE UpdateTreeOnHit(hit_leaf):
    node ← hit_leaf

    WHILE node is not the root:
        parent ← parent(node)

        IF node is the left child of parent:
            parent.bit ← RIGHT
        ELSE:
            parent.bit ← LEFT

        node ← parent
```

```

PROCEDURE SelectVictimOnMiss(root):
    node ← root

    WHILE node is not a leaf:
        IF node.bit indicates LEFT:
            node.bit ← RIGHT
            node ← left child of node
        ELSE:
            node.bit ← LEFT
            node ← right child of node

    RETURN node

```

On a miss, the tree state must be updated **during victim selection exactly as described above**. No additional Tree-PLRU update is required after the victim way is selected.

Any correct internal representation is acceptable, as long as the simulator behavior matches the specification.

Important

For every access that needs to choose a way within a set, the simulator must follow the Tree-PLRU tree to determine the target leaf/way.

Do not first search for an invalid way. Even if some ways in the set are invalid, the selected way is still the one indicated by the procedures above.

Input

The number of ways is guaranteed to be a power of two for every public and hidden testcase.

Each testcase file contains a sequence of memory accesses (a trace file, which records the sequence of memory accesses).

Example of the name of an input file:

```
0_S4_W2_B16.in
```

The meaning of the filename above:

- testcase id = 0
- number of sets = 4
- number of ways = 2
- cache-line size = 16 bytes

Each line of an input file has the following format:

```
{L/S/M} {hex_addr},{bytes}
```

The meaning of data fields:

- **L** : load
- **S** : store
- **M** : modify
- **hex_addr** : the memory address in hexadecimal
- **bytes** : the number of bytes accessed by this memory operation

In the input trace, **M** denotes a modify operation, which semantically corresponds to a load followed immediately by a store.

Example of the content for an input file:

```
L 10,4
S 18,4
L 20,4
S 28,4
S 50,4
L 0,4
S 40,4
L 0,4
M 80,4
L 0,4
S c0,4
M 40,4
L 80,4
S 100,4
```

Output

For each input file, your simulator should generate the corresponding output file.

Example of the output filename, given the input filename `0_S4_W2_B16.in` :

```
0_S4_W2_B16.out
```

Example of the content for the output file:

```
L 10,4 miss
S 18,4 hit
L 20,4 miss
S 28,4 hit
S 50,4 miss
L 0,4 miss
S 40,4 miss
L 0,4 hit
L 80,4 miss eviction
S 80,4 hit
L 0,4 hit
S c0,4 miss eviction
L 40,4 miss eviction
S 40,4 hit
```

```
L 80,4 miss eviction
S 100,4 miss eviction
hits:6 misses:10 evictions:5
PA3$ Bytes Read:          32
PA3$ Bytes Written:       32
PA3$ Read Accesses:       8
PA3$ Write Accesses:      8
PA3$ Read Misses:         6
PA3$ Write Misses:        4
PA3$ Writebacks:          4
PA3$ Miss Rate:           62.500%
```

Judge and Debug

Change the working directory to the `1_cachesim` folder and choose a *make* command to run.

```
$ make judge-all # judge all available testcases
$ make judge-n    # judge all available testcase n, where n = 0, 1, 2
$ make clean      # remove generated files
```

To get the score of your implementation, run `make judge-all` in your environment, and the output of the `make` command will look like this.

```
[PASS] input/0_S4_W2_B16.in
[PASS] input/1_S8_W2_B16.in
[PASS] input/2_S4_W4_B16.in

===== Summary =====
PASS   : 3
WA     : 0
ERR    : 0
SCORE  : 60
```

If your implementation does not pass the result-checking, the output will look like this.

```
[WA] input/0_S4_W2_B16.in
run "git diff --no-index -- answer/0_S4_W2_B16.ans output/0_S4_W2_B16.out" to check.
[WA] input/1_S8_W2_B16.in
run "git diff --no-index -- answer/1_S8_W2_B16.ans output/1_S8_W2_B16.out" to check.
[WA] input/2_S4_W4_B16.in
run "git diff --no-index -- answer/2_S4_W4_B16.ans output/2_S4_W4_B16.out" to check.

===== Summary =====
PASS   : 0
WA     : 3
ERR    : 0
SCORE  : 0
```

You may compare your output against the reference answer with:

```
$ git diff --no-index -- answer/<testcase>.ans output/<testcase>.out
```

Relationship among Problem 1, 2, and 3

The `cachesim.h` and `cachesim.cc` implementation you complete in this problem will be reused in the later problems.

In Problem 2, you will use the same cache simulator framework to simulate memory traces and study cache behavior. In Problem 3, you will integrate your `cachesim.h` and `cachesim.cc` implementation into Spike so that the full MLP inference program can be evaluated with your replacement policy in Spike.

Problem 1 is served as a simpler environment for implementing and debugging the replacement policy before moving on to Spike simulation for performance evaluation.

II. Matrix Transpose Optimization (weight: 40%)

In this problem, you will optimize a matrix transpose implementation to reduce cache misses under a given cache configuration. Your code will first be checked for correctness, and only correct implementations will be evaluated based on cache misses.

Please refer to **Section 2, Cache Optimization Fundamentals** for different optimization techniques, especially for the software-based optimizations. It covers basic concepts. You would explore an effective solution for this problem on your own to obtain better optimization results.

Background

Matrix transpose converts rows into columns and columns into rows. Let $\mathbf{A} = [a_{ij}]$ be an $M \times N$ matrix. The transpose of $\mathbf{A}$, denoted by $\mathbf{A}^T$, is the $N \times M$ matrix defined by $\mathbf{A}^T = [a_{ji}]$. In other words, the element in the i -th row and j -th column of $\mathbf{A}^T$ is the element in the j -th row and i -th column of $\mathbf{A}$. That is, $(\mathbf{A}^T)_{ij} = A_{ji}$, for $1 \leq i \leq n$ and $1 \leq j \leq m$.

In this problem, however, all testcases use **square matrices**. The goal is to minimize cache misses under a fixed cache configuration below while preserving correctness.

Files

```
2_transpose/
├─ answer/          # reference answers
├─ input/           # testcases
├─ output/          # your output
├─ csim_cpp         # an executable which simulates cache stats
├─ errenum.h
├─ evaluate.c       # generates the simulation result
├─ snippet.c        # YOUR WORK HERE
├─ trans.c
├─ trans-driver.c   # driver program for your implementation
├─ checker.py       # check your snippet.c
├─ judge.py
└─ Makefile         # judge commands
```

Important

`csim_cpp` will be copied from `1_cachsim` during the judging of Problem 1, and it is used to simulate the cache misses for this problem, so please make sure you complete Problem 1 before starting this one.

You should **only** modify the following file:

- `2_transpose/snippet.c`

Do **not** modify any other provided files.

Implementation Requirements

Your implementation must correctly compute the transpose of matrix `A` into matrix `B`, i.e., for all valid indices `i` and `j`, the result must satisfy:

```
B[j][i] = A[i][j]
```

You may assume that:

- `A` and `B` refer to distinct matrices, and
- both matrices `A` and `B` are aligned to 4096-byte boundaries.

All testcases use only the following matrix sizes:

- `32x32`
- `64x64`

Cache Configuration

The following cache configuration is used for evaluation of each testcase in Problem 2:

- Number of sets = 16
- Number of ways = 2
- Cache-line size = 32 bytes

Submission Restriction

For this problem, you must submit your `snippet.c` only, as `trans.c` includes the code of `snippet.c`, as shown below.

```
#include <stdint.h>

void trans(int N, int32_t A[N][N], int32_t B[N][N])
{
    int t0, t1, t2, t3, t4, t5, t6, t7, i, j, k, l;
    #include "snippet.c" // snippet.c will be inserted here
}
```

Therefore, your submitted `snippet.c` :

- must not contain any declaration,
- must not change any variable names already provided in `trans.c` ,
- must only use the provided local variables `t0` to `t7` , `i` , `j` , `k` , `l` , and the function parameters `N` , `A` , and `B` .
- You must not introduce any additional local variables, arrays, pointers, structs, helper functions, macros, or typedefs inside the submitted snippet.

Your snippet is implemented in standard C language, and includes loops and conditionals, as long as it fits into the provided function body and obeys the restrictions above.

Note

In `snippet.c` , please do not use RISC-V inline assembly. This problem will not be run in Spike; instead, it uses Valgrind Lackey to generate the program's memory trace, which is then fed into the cache simulator implemented in Problem 1 to simulate the hit, eviction, and miss statistics.

Correctness Checking

For each testcase, correctness is checked first by comparing your generated output against the reference output using `git diff` .

If the transpose result is incorrect, the testcase receives zero points.

The cache misses of your implementation will be evaluated for scoring, only if the correctness check of your implementation is passed.

Evaluation

If your implementation passes correctness checking, the memory-access trace of your transpose code will be evaluated under the cache configuration above.

The score of each testcase depends on the number of cache misses incurred by your implementation.

Note

In the cache simulation, accesses to the provided local variables `t0~t7`, `i`, `j`, `k`, `l`, and the function parameter `N` are not taken into account. You should pay particular attention to complying with the restrictions and to how `A[][]` and `B[][]` are used.

Input

The input files contain square matrices of `int32_t` values.

Two public testcases and two hidden testcases are present in Problem 2.

- Public:
 - 32_0.in
 - 64_0.in
- Hidden:
 - One additional 32x32 testcase: 32_1.in
 - One additional 64x64 testcase: 64_1.in

Output

- The provided code in PA 3 generates the output file using your snippet.c implementation, given a valid input file.
- For the public testcases, the output files are 32_0.out and 64_0.out .
- The hidden testcases follow the same rule.

Judge and Debug

Change the working directory to the 2_transpose folder and choose a make command to run.

```
$ make judge-all # judge all available testcases
$ make judge-n    # judge public testcases n, where n = 32, 64
$ make clean      # remove generated files
```

To get the score of your implementation, run make judge-all in your environment, and the output of the make command will look like this.

```
[PASS] input/32_0.in,
[NAIVE] misses=1170, [IMPROVED] misses=297, score=22.8
[PASS] input/64_0.in,
[NAIVE] misses=4722, [IMPROVED] misses=1544, score=12.01

===== Summary =====
PASS    : 2
WA      : 0
ERR     : 0
SCORE   : 34.81
```

The above shows an example output for the testcase input/32_0.in ,

- [NAIVE] misses=1170 means naive transpose function produces 1,170 cache misses,
- [IMPROVED] misses=297 means trans.c produces 297 cache misses,
- score=22.8 is based on [IMPROVED] misses=297 and **section 5, Scoring Criteria**.

If your implementation is incorrect, the output will look like this.

```
[WA] input/32_0.in
Run "git diff --no-index -- answer/32_0.ans output/32_0.out" to check.
[WA] input/64_0.in
Run "git diff --no-index -- answer/64_0.ans output/64_0.out" to check.

===== Summary =====
PASS      : 0
WA        : 2
ERR       : 0
SCORE     : 0
```

You may compare your output against the reference answer with the following command.

```
$ git diff --no-index -- answer/<testcase>.ans output/<testcase>.out
```

III. MLP Matrix Multiplication Optimization (weight: 70%)

In this problem, you will optimize the matrix multiplication kernel used in a multilayer perceptron inference program, and choose an optimal cache organization for the MLP program. Your implementation must ensure output correctness while improving performance over the baseline implementation.

Please refer to **Section 2, Cache Optimization Fundamentals** for both hardware- and software-based optimization techniques. You would explore an effective solution for this problem to further optimize performance.

Background

A multilayer perceptron is a feed-forward neural network composed of multiple layers of linear transformations and nonlinear activation functions. During inference, one of the most performance-critical operations is matrix multiplication, which is used to combine input activations with weight matrices to produce the outputs of each layer.

Matrix multiplication combines two matrices to produce a third matrix. For matrices A and B , their product matrix C is defined by:

$$C_{ij} = \sum_k A_{ik} \times B_{kj}.$$

As this operation is performed repeatedly and often dominates execution time in MLP inference, optimizing the matrix multiplication kernel can significantly improve overall performance.

For additional background on MLPs, you may refer to [Wikipedia](#) and [But what is a Neural Network?](#).

Simplified Inference Flow

You will be provided with an inference program in PA 3. During the forward pass of a two-layer MLP, the matrix multiplication kernel (`matmul`) is invoked twice. The involved computation kernel

invocations are shown below.

```
/* forward pass
 *
 * input  : NUM_TEST x INPUT_DIM
 * hidden : NUM_TEST x HIDDEN_DIM
 * output : NUM_TEST x OUTPUT_DIM
 *
 * where NUM_TEST=300, INPUT_DIM=284, HIDDEN_DIM=128, OUTPUT_DIM=10
 *
 * Layer 1: hidden = ReLU(input * W1 + b1)
 * Layer 2: output =          hidden * W2 + b2
 */
static void forward(float *input, float *hidden, float *output) {
    /* Layer 1 */
    matmul(input, W1, hidden, NUM_TEST, INPUT_DIM, HIDDEN_DIM);
    add_bias(hidden, b1, NUM_TEST, HIDDEN_DIM);
    relu(hidden, NUM_TEST * HIDDEN_DIM);

    /* Layer 2 */
    matmul(hidden, W2, output, NUM_TEST, HIDDEN_DIM, OUTPUT_DIM);
    add_bias(output, b2, NUM_TEST, OUTPUT_DIM);
}
```

Files

```
3_mlp/
├── answer/           # reference answers
├── input/            # testcases
├── output/           # your output
├── helper.py         # helper for debugging data output
├── matmul_improved.c # YOUR WORK HERE
├── dc_config.py      # YOUR WORK HERE
├── matmul_naive.c
├── mlp_inference.c   # the program which is going to emulate in Spike
├── judge.py
└── Makefile          # judge commands
```

You should **only** modify the following files:

- 3_mlp/matmul_improved.c
- 3_mlp/dc_config.py

Do **not** modify any other provided files.

Implementation Requirements

In PA 3, your implementation must:

1. ensure the functional behavior of the original matrix multiplication kernel,
2. ensure output correctness for every testcase,
3. perform optimizations, such as code vectorization, inside `matmul_improved.c`,

4. avoid modifying the surrounding MLP inference logic or any other provided files,
5. improve performance over the baseline implementation while preserving correctness, and
6. choose a valid data-cache configuration by modifying `dc_config.py` under the constraints described below.

Any submission that produces incorrect outputs for a testcase will receive zero points for that testcase, regardless of performance.

You may apply **software optimizations** within `matmul_improved.c` as long as correctness is preserved and the provided PA 3's code is not modified. Effective solutions will typically require careful attention to memory access patterns, locality, and data reuse.

Your implementation must optimize the given matrix multiplication kernel itself rather than moving the work into unrelated parts of the program.

After correctness is verified, performance will be evaluated against the baseline implementation using the provided judging method.

In this problem, the `cachesim.h` and `cachesim.cc` implementation completed in Problem 1 will be reused as part of the Spike-based evaluation setup. In other words, Problem 3 evaluates both your optimized `matmul_improved.c` and your cache-simulator implementation from Problem 1 together in the full MLP inference program.

Rebuild Your Customized Spike

The example commands for re-building Spike simulator to incorporate with your customized cache simulator are shown below for your reference. On your own machine, you should use these commands accordingly based on where Spike is installed and how your volume is mounted. In our provided container environment, the Spike source code is located at the path shown below. This would also be the environment used in the computer classroom during the examination of PA 3.

```
# copy your implementation into the Spike source directory
$ cp /workspace/l_cachesim/cachesim.h ~/riscv/riscv-isa-sim/riscv/cachesim.h
$ cp /workspace/l_cachesim/cachesim.cc ~/riscv/riscv-isa-sim/riscv/cachesim.cc

# build Spike
$ cd ~/riscv/riscv-isa-sim/build/
$ make
$ make install
```

Build and Execution Environment

This problem is graded using the provided code to build and execution.

In the reference grading setup, the program is compiled and executed with the following settings:

- compilation optimization level: `-O1`
- compilation target: `rv64gc`
- target ISA for Spike simulation: `RV64GCV_Zicntr`

You are encouraged to use RVV in this problem.

Submissions are expected to work correctly under this environment.

Cache Configuration

The default cache configuration (`--dc=16:4:64`) used for evaluation of each testcase is shown below. This default configuration is served as a baseline evaluation of `mlp_inference.c` with `matmul_naive.c`.

- Number of sets = 16
- Number of ways = 4
- Cache-line size = 64 bytes

You will need to choose an optimal cache configuration for evaluating `mlp_inference.c` with `matmul_improved.c` by modifying `dc_config.py`. The cache configuration must satisfy the following constraints:

- Total size of cache is less than or equal to 4,096 bytes.
- Number of sets > 1
- Number of ways is power of 2
- Cache-line size > 8 and it is power of 2

Correctness Check

For the matrix multiplication code, the output correctness is checked **element-wise** against the reference output.

Your output is considered correct if every output element satisfies the following constraint:

$$|ref - stu| \leq atol + rtol \cdot |ref|,$$

where

$$atol = 0.001, \quad rtol = 0.001$$

Here, ***ref*** denotes the reference element, and ***stu*** denotes the corresponding element produced by a student's (your) program.

It is important to note that associativity does not hold for floating-point operations, such as additions and multiplications, and your optimized code version may result in different numerical results due to differences in the computation order. The constraint above is to account for such

small variations in the numerical results. In typical cases, a correct matrix multiplication implementation should pass this correctness check.

Evaluation

If your implementation passes the correctness check above, the full MLP inference program using your `matmul_improved.c` will be used for performance evaluation.

During evaluation, the program is run by Spike with the cache simulation enabled. The simulated data-cache behavior of your implementation is used to estimate memory subsystem overhead (**MemCycle**), and instruction cycles are collected (**InstCycle**) through the performance counters from `Zicntr`, as described in **Section 3, Performance Evaluation Methodology**. The scores you earned will be based on the measured **Improvement Ratio**.

As the delivered performance represents the combined effect on both CPU execution pipeline and the memory sub-system, effective solutions will usually require careful attention to memory access patterns, locality, and data reuse for memory performance optimizations and to calculations for CPU execution optimizations.

Note

The reported statistics represent an approximation of execution time for the modelled processor. The performance data may or may not match the data collected on a real hardware platform.

In particular, the measured statistics come from executing the entire `mlp_inference.c` program under Spike, and therefore these performance data may also include time spent on the codes other than the matrix multiplication kernel, such as proxy-kernel (`pk`) startup/initialization and program I/O.

In our setup, the matrix multiplication workload is expected to dominate the overall execution time. Therefore, improving the matrix multiplication kernel should still have a noticeable impact on the overall performance.

Input

In this problem, the testcase data are embedded directly in C source files. That is, each testcase is provided as a `.c` file that contains the `float32` input activations used by the MLP inference program.

Two testcases are used in this problem.

- Public:
 - `data0.c`

- Hidden:

- one additional testcase: `data1.c`

Each testcase corresponds to an input matrix of shape `300 x 784 ( NUM_TEST x INPUT_DIM )`.

Output

The output file generated after the Spike simulation on your `matmul_improved.c` implementation will be like below.

Public output:

- `mlp0.out`

Hidden output:

- `mlp1.out`

Each output matrix has shape `300 x 10 ( NUM_TEST x OUTPUT_DIM )`.

Correctness is determined by the element-wise tolerance rule described in **Problem 3 Correctness Check** above.

Judge and Debug

Change the working directory to the `3_mlp` folder and choose a make command to run.

```
$ make judge-all # judge available testcases
$ make clean      # remove generated files
$ make print_data # print your output in human-readable
```

To get the score of your implementation, run `make judge-all` in your environment, and the output of the make command will look like this.

```
===== Original version =====
inst. cycles: 214406569
D$ Bytes Read:          249925401
D$ Bytes Written:       2550035
D$ Read Accesses:      61812472
D$ Write Accesses:     371435
D$ Read Misses:        30644654
D$ Write Misses:       72572
D$ Writebacks:         81309
D$ Miss Rate:          49.397%
Memory subsystem access overhead = 3103189281 (cpu cycle)
Instruction cycles = 214406569 (cpu cycle)
-----
===== Improved version =====
inst. cycles: 146396925
D$ Bytes Read:          165013453
D$ Bytes Written:       8815431
```

```

D$ Read Accesses:      40330516
D$ Write Accesses:     1691437
D$ Read Misses:        483220
D$ Write Misses:       37235
D$ Writebacks:         169057
D$ Miss Rate:          1.239%
Memory subsystem access overhead = 93546998 (cpu cycle)
Instruction cycles = 146396925 (cpu cycle)
-----
Output Correctness: Pass
max_abs_err = 1.144e-05
memory access overhead improved ratio: 33.1725159261658
total overhead improved ratio: 13.826546671907169

===== Summary =====
PASS    : 1
WA      : 0
ERR     : 0
SCORE   : 6.4

```

The above shows an example output for a testcase:

- `Memory subsystem access overhead` corresponds to *MemCycle*.
- `Instruction cycles` corresponds to *InstCycle*
- `total overhead improved ratio` corresponds to **Improvement Ratio**
- `max_abs_err` is the maximum absolute difference between the student's output and the reference output among all elements in the testcase.

If your implementation is incorrect, the output will look like this.

```

===== Original version =====
inst. cycles: 214406569
D$ Bytes Read:      249925401
D$ Bytes Written:    2550035
D$ Read Accesses:    61812472
D$ Write Accesses:   371435
D$ Read Misses:      30644654
D$ Write Misses:     72572
D$ Writebacks:       81309
D$ Miss Rate:        49.397%
Memory subsystem access overhead = 3103189281 (cpu cycle)
Instruction cycles = 214406569 (cpu cycle)
-----
===== Improved version =====
inst. cycles: 146383803
D$ Bytes Read:      165013461
D$ Bytes Written:    8789151
D$ Read Accesses:    40330518
D$ Write Accesses:   1684870
D$ Read Misses:      483212
D$ Write Misses:     37233
D$ Writebacks:       169008
D$ Miss Rate:        1.239%
Memory subsystem access overhead = 93539443 (cpu cycle)
Instruction cycles = 146383803 (cpu cycle)
-----

```

```
Output Correctness: Fail
first mismatch at sample=0, class=0
ref=-6.671, out=-10.749, abs_err=4.078e+00
```

```
===== Summary =====
PASS      : 0
WA        : 1
ERR       : 0
SCORE     : 0
```

You may compare your output against the reference answer with:

```
$ make print_data
```

This command prints the generated and reference output data in a human-readable form for inspection.

5. Scoring Criteria

PA 3 contains three problems. Each problem is scored from **0 to 100**, denoted as P_i . The final score for PA 3 is computed as a weighted sum of the three problem scores.

$$\text{TotalScore} = 0.1 \cdot P_1 + 0.4 \cdot P_2 + 0.7 \cdot P_3$$

where:

- P_1 is the normalized score of Problem 1,
- P_2 is the normalized score of Problem 2, &
- P_3 is the normalized score of Problem 3.

Therefore, the maximum total score is:

$$0.1 \times 100 + 0.4 \times 100 + 0.7 \times 100 = 120.$$

Problem 1 Score (weight: 10%)

Problem 1 contains 5 testcases in total, and each testcase is worth **20 points**.

- 3 public testcases account for 60 points.
- 2 hidden testcases account for 40 points.

A testcase earns its 20 points only if your implementation produces the correct result for that testcase. Otherwise, the score for that testcase is 0.

The Problem 1 score P_1 is obtained by the sum of the earned scores for all testcases.

Problem 2 Score (weight: 40%)

Problem 2 contains 4 testcases in total, and each testcase is worth **25 points**.

- The 2 public testcases account for 50 points.
- The 2 hidden testcases account for 50 points.

For each testcase, your program must first produce the correct transposed matrix. If a testcase fails correctness checking, its score for that testcase is 0. When the transposed result is correct, the score for each testcase is determined by the number of cache misses. Let m be the number of cache misses for a testcase of size n , where $n \in \{32, 64\}$, and let the full score for that testcase be **25**. Then the testcase score is defined as follows.

$$\text{score}(n, m) = \begin{cases} 25, & m \leq L_n, \\ 0, & m \geq U_n, \\ 25 \left(\frac{U_n - m}{U_n - L_n} \right)^{0.5}, & L_n < m < U_n. \end{cases}$$

where the lower and upper miss thresholds are

$$(L_{32}, U_{32}) = (256, 500), \quad (L_{64}, U_{64}) = (1024, 1700).$$

For $L_n < m < U_n$, the computed testcase score is rounded to two decimal places.

where:

- n is the matrix size of the testcase. In this problem, $n \in \{32, 64\}$.
- m is the number of cache misses produced by the student's transpose implementation on that testcase.
- L_n is the lower miss threshold for matrix size n . If $m \leq L_n$, the testcase receives full score.
- U_n is the upper miss threshold for matrix size n . If $m \geq U_n$, the testcase receives 0 points.
- L_{32} and U_{32} are the lower and upper miss thresholds for the 32-by-32 testcase.
- L_{64} and U_{64} are the lower and upper miss thresholds for the 64-by-64 testcase.

Problem 3 Score (weight: 70%)

Problem 3 contains 2 testcases in total.

- The public testcase contributes **10 points**.
- The hidden testcase contributes **90 points**.

For each testcase, your program must first pass correctness checking. If a testcase fails correctness checking, its score for that testcase is 0. When the output is correct, the score for each testcase is determined by the **Improvement Ratio** r over the baseline implementation. The **Improvement Ratio** is described in **Section 3, Performance Evaluation Methodology**.

Let r be the **Improvement Ratio** for a testcase, and let the full score for that testcase be S , where $S \in \{10, 90\}$.

The testcase score is defined as below.

$$\text{score}(r, S) = \begin{cases} 0, & r \leq L, \\ S, & r \geq U, \\ S \left(\frac{r - L}{U - L} \right)^{0.5}, & L < r < U, \end{cases}$$

where:

- r is the **Improvement Ratio** of the student's implementation over the baseline implementation for the testcase.
- S is the full score of the testcase. In this problem, $S = 10$ for the public testcase and $S = 90$ for the hidden testcase.
- L is the lower improvement threshold. If $r \leq L$, the testcase receives 0 points.
- U is the upper improvement threshold. If $r \geq U$, the testcase receives full score.

The improvement thresholds are:

$$L = 4, \quad U = 28.$$

For $L < r < U$, the computed testcase score is rounded to two decimal places.

The resulting normalized Problem 3 score is then used in the final weighted sum of the assessment.

6. Test by Judge Program

You can use the provided top-level Makefile commands to test your code locally. The command `make judge-1` is for Problem 1, and the command `make judge-all` to for building all three problems.

```
$ make judge-1
$ make judge-2
$ make judge-3
$ make judge-all
```

If you want to clean the generated files, you can use the following commands.

```
$ make clean-1
$ make clean-2
$ make clean-3
$ make clean-all
```

Notes on Failed Tests

There are three common errors, compilation error, runtime error, and wrong answer error. They are illustrated as follows.

Compilation Error (CE): your submitted source file does not compile.

```
[CE] See message below
In file included from trans.c:7:
snippet.c: In function 'trans':
snippet.c:12:33: error: expected ';' before 'B'
    12 |             B[j][i + k] = t0
        |                               ^
    13 |             B[j+1][i + k] = t1;
        |             ~
make[1]: *** [Makefile:17: trans-driver] Error 1

===== Summary =====
PASS      : 0
WA        : 0
ERR       : 1
SCORE     : 0
```

Runtime Error (RE): your program crashes or terminates abnormally during execution.

```
[RE] input/32_0.in, status=139
Segmentation fault (core dumped)
[RE] input/64_0.in, status=139
Segmentation fault (core dumped)

===== Summary =====
PASS      : 0
WA        : 0
ERR       : 2
SCORE     : 0
```

Wrong Answer (WA): your program runs successfully but the output does not match the expected result.

```
[WA] input/32_0.in
Run "git diff --no-index -- answer/32_0.ans output/32_0.out" to check.
[WA] input/64_0.in
Run "git diff --no-index -- answer/64_0.ans output/64_0.out" to check.

===== Summary =====
PASS      : 0
WA        : 2
ERR       : 0
SCORE     : 0
```

7. Programming Assessment Guidelines

Rules

- Please bring your student ID card as proof of identity.
- During the exam, only NCKU Moodle may be used for downloading the exam files and uploading your submission.

- The use of USB or external devices, cheat sheets, paper notes, or any AI tools is strictly prohibited.
- Violations of the above rules will be regarded as cheating, and the exam score will be recorded as 0.
- There will be no hints provided during the exam.

Submission of Your Code

Submit your zip file to NCKU Moodle.

The zip file organization of your directory must be as follows:

```
C0_F12345678_PA3.zip
├── C0_F12345678_PA3
│   ├── cachesim.cc
│   ├── cachesim.h
│   ├── dc_config.py
│   ├── matmul_improved.c
│   └── snippet.c
```

Important Notes

- Replace `C0_F12345678_PA3` with your own student ID, for example, `C0_E64112287_PA3`.
- You must submit **exactly** the following files.
 - `cachesim.cc`
 - `cachesim.h`
 - `snippet.c`
 - `matmul_improved.c`
 - `dc_config.py`
- Do not submit any files or folders other than the above.
- Incorrect format, including wrong filename, wrong directory structure, or unexpected files, will result in a 10-point deduction.
- Unexpected file contents that prevent the judge from continuing may result in a score of 0.
- Do not modify files other than the four submitted files listed above.

8. References

- [CS 3330 - Tree-PLRU](#)
- [RISC-V V Intrinsics Specification \(PDF\)](#)
- [RISC-V ISA Manual \(PDF\)](#)
- [An Anatomy of Optimized Matrix Multiplication in AArch64](#)